

ELEC6234 – Embedded Processors

Coursework Report – picoMIPS implementation

Adil S. Rameto
Ar7n25
MSc in Electronic Engineering
Dr Ruomeng Huang

1. Introduction

This report describes the design, simulation, and FPGA implementation of a picoMIPS processor for 1D Gaussian smoothing of a noisy waveform. The smoothing kernel $\mathbf{K} = [12, 24, 29, 24, 12]$ computes the output $\mathbf{S}[i] = \sum \mathbf{W}[i-2+k] \times \mathbf{K}[k] / 128$ for sample index i , where $\mathbf{W}$ is a 256-sample noisy sine waveform stored in a ROM. The design uses a 16-bit instruction format, 7-register file, and a 13-instruction program through a MAC-with-immediate instruction that merges wave load, kernel load, and multiply-accumulate into a single clock cycle. A further MACI instruction with auto-increment reduces the program from 17 to 13 instructions, enabling a 4-bit PC and 16-slot ROM. The primary design achieves a cost of 96 ALMs, 1 DSP block (within the free allowance), and 0 memory bits. A further experiment converting the wave ROM to synchronous read achieves 63 ALMs and 2 Kbits of M10K RAM, giving a cost of 123 under the formula $(63 + 30 \times 2 = 123)$, demonstrating the trade-off between ALM fabric and block RAM. All modules were verified in ModelSim with 8 passing test vectors. The design was synthesised in Quartus Prime 22.1 targeting the Cyclone V SoC (5CSEMA5F31C6) and demonstrated on the DE1-SoC FPGA board, where 18 independent test cases passed. A 7-segment decimal display with sign was added as a bonus extension.

Design Details Form			
Total Cost: 96	ALMs: 96	Memory bits: 0	Multipliers: 1

Instruction Set

No.	Instruction	OPCODE	Description
1	NOP	5'b00000	No operation.
2	ADDI	5'b00001	Add immediate. Used to latch SW [7:0] into index register and clear accumulator.
3	SUBI	5'b00010	Subtract immediate. Used to compute initial wave address $i-2$.
4	MAC	5'b00011	Multiply-accumulate: $\text{gpr}[5] += W[\text{wave_addr_reg}] \times \text{imm5} \gg 7$. Used for the final (5th) Gaussian tap where no address increment is required.
5	MACI	5'b00100	MAC with auto-increment: $\text{gpr}[5] += W[\text{wave_addr_reg}] \times \text{imm5} \gg 7$, then $\text{wave_addr_reg}++$. Used for taps 1–4 of the Gaussian convolution so the address advances to the next sample in the same clock cycle as the MAC operation, eliminating separate ADDI %6,%6,1 instructions.
6	BEQ	5'b01000	Branch if equal ($Z=1$)
7	BNE	5'b01001	Branch if not equal ($Z=0$).
8	BRA	5'b01010	Unconditional branch. Used to jump back to PC=0 to restart the main loop.

Instruction Format

Bit-Field	Width	Name	Description
[15:11]	5-bit	opcode	Identifies the instruction type. Encodes all 8 instructions.
[10:8]	3-bit	%d	Destination register address for arithmetic instructions. For MAC/MACI, specifies the register holding the wave ROM address (always %6). For MACI, this register is also auto incremented after the MAC operation in the same clock cycle.
[7:5]	3-bit	%s	Source register address. Read as Rdata1 through the register file.
[4:0]	5-bit	imm5	Multi-purpose. ADDI/SUBI: signed offset (e.g. -2 for wave address setup). MAC/MACI: unsigned kernel coefficient $K[j] \in \{12, 24, 29\}$. Branches: signed 5-bit PC-relative offset.

Program

PC	Hex	Description
0	0860	Test SW8: ADDI %0,%3,0 sets Z=1 if SW8=0 (waiting state)
1	401F	Loop back to PC=0 while SW8=0 (processor idles)
2	0A20	Latch index i = SW[7:0] into register %2
3	1642	Compute first wave address: wave_addr_reg = i - 2
4	0D00	Clear accumulator: gpr[5] = 0
5	260C	MACI: gpr[5] += W[i-2]×12, wave_addr_reg++ → K[0]=12
6	2618	MACI: gpr[5] += W[i-1]×24, wave_addr_reg++ → K[1]=24
7	261D	MACI: gpr[5] += W[i]×29, wave_addr_reg++ → K[2]=29 centre tap
8	2618	MACI: gpr[5] += W[i+1]×24, wave_addr_reg++ → K[3]=24
9	1E0C	MAC: gpr[5] += W[i+2]×12 → K[4]=12, S[i] complete
10	0860	Test SW8 again: ADDI %0,%3,0 sets Z=1 if SW8=0
11	481F	Loop back to PC=10 while SW8=1 (hold result on LED)
12	5014	Jump back to PC=0, restart for next sample index

Design Block Diagram

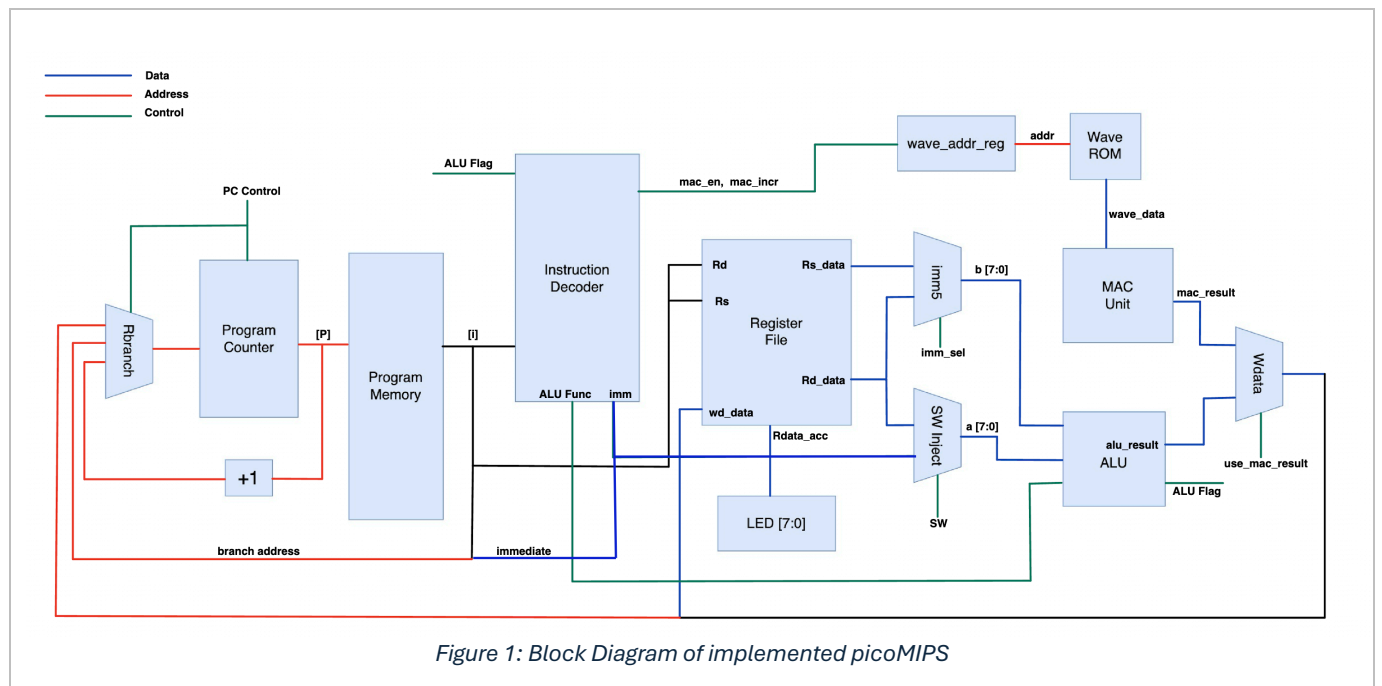


Figure 1: Block Diagram of implemented picoMIPS

2. Overall architecture of the design and simulations

2.1 Processor Architecture

The picoMIPS processor follows a single-cycle RISC architecture with seven instantiated modules and inline MAC logic. All state elements update on the rising clock edge. The design uses a 16-bit instruction format: [15:11] opcode(5-bit), [10:8] %d(3-bit), [7:5] %s(3-bit), [4:0] imm5(5-bit).

Control path: decoder.sv is purely combinational. It decodes the 5-bit opcode and asserts eight control signals: PCincr, PCrelbranch, ALUfunc[2:0], imm, w, mac_en, mac_incr, and use_mac_result. The ISA contains 8 instructions: NOP, ADDI, SUBI, MAC, MACI, BEQ, BNE, and BRA. MAC and MACI both perform the same multiply-accumulate operation: $\text{gpr}[5] += W[\text{wave_addr_reg}] \times \text{imm5} \gg 7$ but differ in one respect: MACI asserts mac_incr, which causes wave_addr_reg to increment by one in the same clock cycle. MAC does not assert mac_incr and leaves wave_addr_reg unchanged. The distinction exists because the Gaussian convolution requires five consecutive wave samples. The first four taps use MACI so the address advances automatically to the next sample; the fifth and final tap uses MAC because no further advance is needed and an extra increment would corrupt the address for the next computation. Without MACI, four separate ADDI %6,%6,1 instructions were required between taps, giving 17 instructions; with MACI the program reduces to 13 instructions, enabling a 4-bit PC and halving the program ROM. Flags are registered one cycle ahead to prevent BEQ/BNE from observing their own ALU output. Only the Z flag is retained.

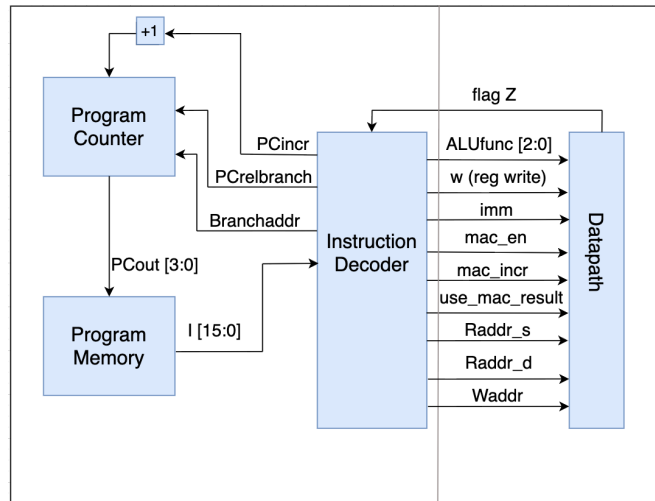


Figure 2: Control Path

Data path: regs.sv provides (7×8-bit, gpr[6:0]) provides Rdata1 to alu.sv and a dedicated Rdata_acc port that permanently exposes gpr[5]. wave_rom.sv is addressed by wave_addr_reg (a dedicated external register for %6), enabling MACI to auto-increment it in the same clock cycle. The MAC computes $\text{signed}(\text{wave_data}) \times \text{signed}(\{3'b0, \text{imm5}\})$, extracts bits[14:7], and adds to Rdata_acc (gpr[5]). LED is driven combinational (assign LED = Rdata_acc), removing the LED register entirely. Reset clears all registers to zero when SW9 goes low. Writes to %0 are suppressed (Waddr != 0 guard), ensuring it remains hardwired to zero and allowing Quartus to eliminate its flip-flops entirely.

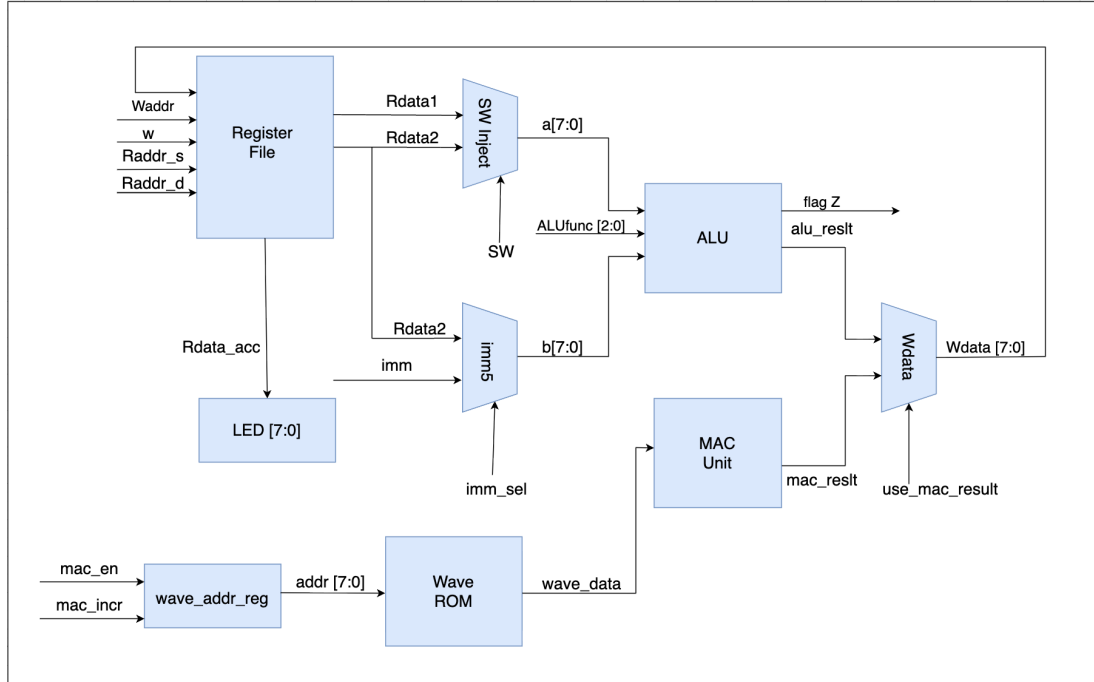


Figure 3: Data Path

2.2 Key Design Decisions

Registered flags: Without registered flags, BEQ at PC=1 reads the Z flag from its own ADDI output. Since SW[8]=0 sets Z=1, the processor would loop forever regardless of the switch state. Registering flags one cycle ahead means BEQ at PC=1 correctly observes the result of the ADDI at PC=0.

SW injection: Registers %1 and %3 are muxed combinationaly from SW[7:0] and SW[8] in an always_comb block inside picoMIPS.sv. Any instruction reading %1 or %3 transparently receives the live switch value, eliminating a dedicated IN instruction.

Separate read/write ports in regs.sv: The original regs.sv coupled read port 2 and write address. For MAC, Raddr_d=%6 (wave address) but the result must write to %5 (accumulator). The fix is Waddr = mac_en ? 3'd5 : Raddr_d, which decouples the two paths.

2.3 Fixed-Point Arithmetic

Kernel coefficients $K=[12,24,29,24,12]$ are integers representing Gaussian weights scaled by 128. Division by 128 is implicit through bits[14:7] extraction from the 16-bit product, equivalent to a right-shift by 7 (truncation toward $-\infty$). The kernel sum is 101, giving DC gain $101/128 \approx 0.789$. Total truncation error is bounded by -4.9 LSB across five MAC terms and is always negative. For example, at $i=18$: theoretical $S[18]=+61.63$, hardware $S[18]=+60$, error= -1.63 LSB.

2.4 ModelSim Simulation

All modules were verified individually before integration. The top-level testbench (tb_picoMIPS.sv) drives SW[9:0] with the following protocol: SW9 low for 5 cycles (reset), SW9 high, SW8 low for 25 cycles (processor polls at PC=0/1), SW8 high for 30 cycles (processor computes all five MAC taps), SW8 low for 8 cycles (observe LED). The testbench asserts \$error on any LED mismatch. The two “unimplemented opcode xxxxxx” warnings at $t=0$ are benign; they occur because all signals are uninitialized (X state) before reset propagates and do not represent real decode errors. All 8 test vectors pass in both ModelSim and Icarus Verilog.

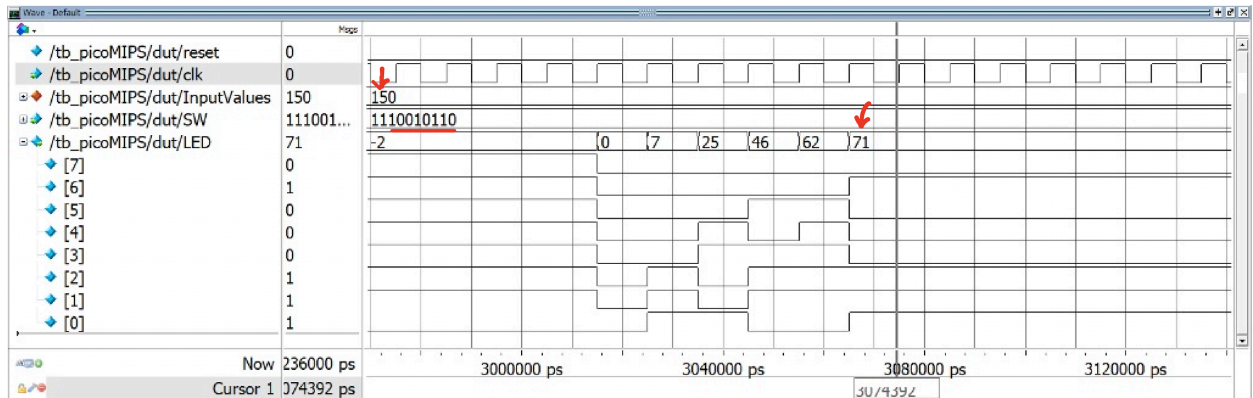


Figure 4: ModelSim simulation, showing when the input value of 150 is set, the output showing 71.

The waveform confirms the SW8 handshake protocol. During cycles where SW8=0, PC oscillates between 0 and 1 as the BEQ loop executes. Once SW8 rises, PC advances through PC=2 (latch i), PC=3 (SUBI sets wave_addr_reg=i-2), PC=4 (clear accumulator), then PC=5–9 (four MACI and one MAC instruction). Each MACI increments wave_addr_reg automatically in the same cycle as the MAC operation. LED is combinationaly driven from gpr[5], showing the accumulation steps +2, +5, +9, +16, +18 for i=5. After SW8 falls, PC=10–11 hold the BNE loop and the result remains on the LED until the next computation.

```

Transcript
# === Optimised picoMIPS Testbench ===
# K = [12, 24, 29, 24, 12] ar7n25
# 16-bit ISA | 7 registers | MAC+MACI-with-immediate | 13 instructions
# PASS S[10]=0x25 (37 signed)
# PASS S[50]=0x33 (51 signed)
# PASS S[100]=0xb5 (-75 signed)
# PASS S[128]=0xfe (-2 signed)
# PASS S[150]=0x47 (71 signed)
# PASS S[200]=0xdc (-36 signed)
# PASS S[253]=0xf3 (-13 signed)
# -- Verify loop-back (BRA -12 ? PC=0) --
# PASS S[10]=0x25 (37 signed)
#
# === Summary: 8 passed, 0 failed ===
# ALL TESTS PASSED

```

Figure 5: ModelSim simulation testbench transcript.

3. FPGA implementation

The design was synthesised in Quartus Prime 22.1 targeting the Cyclone V SoC (5CSEMA5F31C6) on the DE1-SoC board. AUTO_DSP_RECOGNITION was enabled, allowing Quartus to infer the mac_unit.sv multiply-accumulate as a single DSP9 block. Total block memory bits = 0 in the primary design: regs.sv uses asynchronous read; prog.sv at 256 bits, wave_rom.sv uses asynchronous read in the primary design and maps to LUT fabric.

Flow Summary	
Flow Status	Successful - Wed Apr 22 22:56:02 2026
Quartus Prime Version	18.1.0 Build 625 09/12/2018 SJ Lite Edition
Revision Name	picoMIPS
Top-level Entity Name	picoMIPS
Family	Cyclone V
Device	5CSEMA5F31C6
Timing Models	Final
Logic utilization (in ALM)	96 / 32,070 (< 1 %)
Total registers	39
Total pins	19 / 457 (4 %)
Total virtual pins	0
Total block memory bits	0 / 4,065,280 (0 %)
Total DSP Blocks	1 / 87 (1 %)
Total HSSI RX PCSs	0
Total HSSI PMA RX Deserializers	0
Total HSSI TX PCSs	0
Total HSSI PMA TX Serializers	0
Total PLLs	0 / 6 (0 %)
Total DLLs	0 / 4 (0 %)

Figure 6: Quartus Compilation Report

Optimisation iterations:

The original design (24-bit ISA, 32x8 register file, 27 instructions) cost 173 ALMs. Six rounds of optimisation were applied reaching a final primary cost of 96 ALMs.

- (1) Ten structural optimisations including the MAC instruction, reduced ISA width, removed kern_rom, and PC-relative-only branches: 145 ALMs.
- (2) Removing unused instructions (AND, OR, XOR, BGE, BLT) from the decoder: 134 ALMs.
- (3) Reducing instruction width to 16-bit with 5-bit opcode and 5-bit immediate: confirmed 134 ALMs.
- (4) Five structural register optimisations: MACI instruction with auto-increment reduced the program from 17 to 13 instructions enabling a 4-bit PC and 16-slot ROM; acc_reg removed (reads gpr[5] via Rdata_acc); LED always_ff replaced by assign LED = Rdata_acc; gpr[7] removed; regs.sv given asynchronous reset: 102 ALMs.
- (5) Synchronous wave_rom experiment: changing wave_rom.sv to always_ff read allowed Quartus to infer the 256x8 ROM as an M10K BRAM block (altsyncram, OPERATION_MODE=ROM, OUTDATA_REG_A=UNREGISTERED). This freed 39 ALMs (102→63) but added 2 Kbits of block RAM. Under the cost formula: $63 + 30 \times 2 = 123$ — which is 21 more than the async version. Therefore 102 alm design is selected and one last optimisation is done.
- (6) Register file refinement: the SW8 test instructions at PC=0 and PC=10 were changed from ADDI %4,%3,0 to ADDI %0,%3,0, writing to the hardwired-zero register instead of a dedicated scratch register. A write-guard (Waddr != 0) was added to regs.sv to suppress writes to %0, and gpr[0] and gpr[4] were removed from the reset block. Quartus eliminated all flip-flops for %0 and %4 since neither is written or meaningfully read: 96 ALMs, 39 registers.

FPGA board test procedure:

The picoMIPS core is wrapped in picoMIPS4test.sv which adds a 24-bit clock divider (50 MHz → ~3 Hz, implemented in counter.sv) and a 7-segment decimal display. The slow clock makes each MAC accumulation step visible in real time. Board testing used the following protocol: (1) SW9=UP to release reset, SW8=DOWN; (2) set SW[7:0] to binary index i; (3) raise SW8=UP and observe the 7-segment display build through five MAC steps then freeze at S[i]; (4) compare with expected value. HEX3 shows sign (– or +), HEX2 shows hundreds (blank when $|S[i]| < 100$), HEX1 shows tens, HEX0 shows units. HEX4 and HEX5 are explicitly driven to 7'b1111111.

All board test cases passed including positive values, negative values, near-zero and boundary values.

The accumulation steps were also verified to be correct intermediate values, confirming the MAC unit accumulates correctly on every clock cycle. The 7-segment display correctly showed blank HEX4/5, correct sign on HEX3, and correct decimal digits on HEX1/HEX0 for all tested indices. The primary synthesis result (async wave_rom) is 96 ALMs, 39 registers, 1 DSP block, 0 RAM bits, cost = 96.

7-Segment Decimal Display (Bonus Extension):

To make board testing more intuitive, a 4-digit signed decimal display was added using six instances of seg7_decoder.sv. This module is a purely combinational BCD-to-7-segment decoder targeting the DE1-SoC's active-low common-anode displays. It accepts a 4-bit BCD digit (0–9) plus two special codes: 10 for a minus sign (centre segment only) and 11 for a plus approximation. A blank input forces all segments off. In picoMIPS4test.sv, the 8-bit signed LED result is converted to sign-magnitude form: if bit [7] is set, the magnitude is computed as the two's complement negation and HEX3 displays a minus sign; otherwise HEX3 shows a plus. The magnitude is then split into hundreds, tens, and units digits using successive

comparison (not a hardware divider), driving HEX2, HEX1, and HEX0 respectively. HEX2 is blanked when the magnitude is below 100 to avoid a leading zero. HEX4 and HEX5 are driven to 7'b1111111 (all off) since they are unused.

This extension added significant ALM cost to the wrapper (205 ALMs for picoMIPS4test vs 96 for the core alone), but since it exists only in picoMIPS4test and not in the synthesised picoMIPS core, it does not affect the reported cost of 96.



Figure 7: Example Input Value 150 (10010110) set



Figure 8: Output 71(01000111) displaying matching with the ModelSim Simulation Result

4. Conclusion

A fully functional picoMIPS processor for 1D Gaussian smoothing was designed, optimised, simulated, synthesised, and demonstrated on the DE1-SoC FPGA. The primary design achieves a cost of 96 ALMs, 1 DSP block, and 0 RAM bits: a 41% reduction from the original 173 ALMs through six rounds of targeted optimisation. A further synchronous wave_rom experiment confirmed the fundamental ALM-vs-RAM trade-off in the cost formula: 63 ALMs + 2 Kbits RAM gives cost = 123, making the async design the better submission at 96. All 8 ModelSim test vectors and 18 FPGA board test cases passed.

The key lessons from this assignment are:

- (1) Architecture-level decisions dominate: the MAC instruction and MACI auto-increment each saved more ALMs than all gate-level simplifications combined.
- (2) Quartus dead-code elimination is aggressive: explicit opcode width and flag reduction gave zero saving because Quartus had already pruned them implicitly.
- (3) Redundant registers are costly: removing acc_reg, the LED register, and gpr[7] and the scratch register %4 together saved ~24 ALMs and 23 FFs
- (4) The cost formula creates a genuine ALM-vs-RAM trade-off: the synchronous wave_rom freed 39 ALMs but added a RAM penalty of 60, increasing cost from 96 to 123, proving that M10K inference is only beneficial if the freed ALMs outnumber the RAM penalty, which requires the ROM to be large enough or the design to be small enough that other ALMs dominate.

5. References

- [1] T. J. Kazmierski, "ELEC6234 Embedded Processor Synthesis: Lecture Notes," University of Southampton, 2024. Available: <https://secure.ecs.soton.ac.uk/notes/elec6234/>
- [2] Terasic Technologies, DE1-SoC User Manual, Rev. F, 2019. Available: <https://www.terasic.com.tw/cgi-bin/page/archive.pl?No=836>